KOENIG

step forward



Spring Data JPA and MySQL

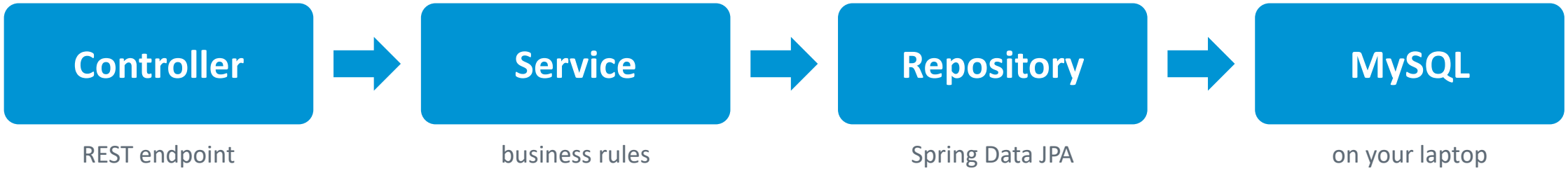
Spring Boot Fundamentals

What You Will Learn

1. ORM and the JPA stack
2. MySQL and connection
3. Entities and mapping
4. Layered API and CRUD
5. Queries, paging, sorting
6. Validation, problem details
7. Transactions and Caching

Demo Scenario: The Catalogue Gets a Database

Until now Learnly's courses lived in memory and disappeared at every restart. We store them in MySQL behind a layered REST API and let Spring Data JPA write most of the SQL.



What can go wrong

- **Half-done work**
- **Wrong settings**
- **Slow reads**

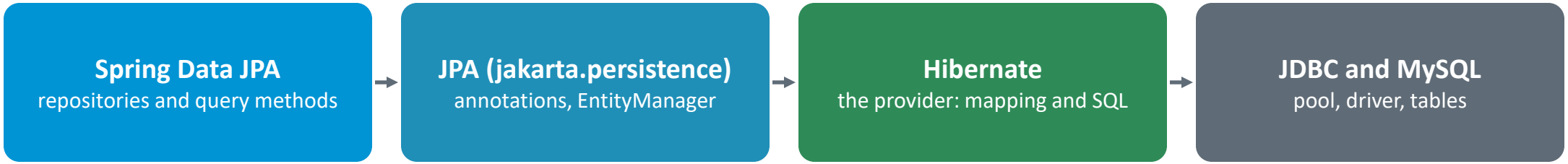
Why ORM?

	Java object	MySQL table
Unit	a Course object	a row of courses
Fields	title, status, launchDate	columns title, status, launch_date
Identity	a reference	the primary key id
Reading	course.getTitle()	SELECT title FROM courses WHERE id = ?

JDBC by hand: prepare a statement, set every parameter, read every column of every row into an object.
ORM: describe the mapping once; the framework writes and runs the SQL.

Objects and tables describe the same data in two different ways

JPA, Hibernate and Spring Data JPA



You write: an entity class and a repository interface.
Spring Boot configures: the DataSource, Hibernate and the repository implementations.
Hibernate sends: the SQL to MySQL through the driver.

Three layers: the standard, the engine and the convenience

What the Project Needs

```
implementation("org.springframework.boot:spring-boot-starter-webmvc")
implementation("org.springframework.boot:spring-boot-starter-data-jpa")
runtimeOnly("com.mysql:mysql-connector-j")
```

Spring Boot builds	from
DataSource	spring.datasource.*
Hibernate	spring.jpa.*
Repositories	your interfaces

Wizard: tick Spring Web, Spring Data JPA and MySQL Driver in New Project > Spring Boot.
Runtime driver: your code never imports it, so it is runtimeOnly.

Three dependencies and Spring Boot does the rest

Connecting Spring Boot to MySQL

```
spring.datasource.url=${LEARNLY_DB_URL:jdbc:mysql://localhost:3306/learnlydb}
spring.datasource.username=${LEARNLY_DB_USER:learnly}
spring.datasource.password=${LEARNLY_DB_PASSWORD:learnlypass}
```

Message	Usually means	Check
Failed to configure a DataSource	no url	the property name
Access denied for user	wrong user or password	mysql-setup.sql, the password
Unknown database	the schema does not exist	CREATE DATABASE
Communications link failure	MySQL is not running or the port is wrong	the MySQL84 service, the port

Three properties build the DataSource

ddl-auto and SQL Logging

ddl-auto	At start-up Hibernate	Use
none	does nothing	production with migrations
validate	checks that the tables fit the entities	production
update	adds missing tables and columns	learning, never production
create / create-drop	drops and creates the tables	tests and throwaway data

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Let Hibernate create the table while you learn, and look at the SQL

Demo: Connect to MySQL

What we do

- **Start:** with datasource settings.
- **Connect:** url, user and password.
- **Break:** a wrong password.

What it shows

- **Three properties:** build the DataSource.
- **A clear error:** which property is missing.
- **Read the cause:** the java.sql Caused by line.

Entities: Classes Mapped to Tables

```
@Entity
@Table(name = "courses")
public class Course {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 120)
    private String title;

    protected Course() { }
}
```

@Entity: persistent class.
@Table: the table name.
@Id: the primary key.
@Column: nullable and length.
No-arg constructor: required (protected is fine).

A few annotations tell Hibernate what the table looks like

Identifiers and Generation

Strategy	How the id appears	On MySQL
IDENTITY	AUTO_INCREMENT column, read back after the INSERT	the normal choice
SEQUENCE	a database sequence	not available (emulated with a table)
UUID	a random value created by Hibernate	works; larger index
assigned	you set the id yourself	natural keys only

Use Long, not long: a null id means the object is new and not yet in the table.

Mapping Types and Columns

```
@Enumerated(EnumType.STRING)
@Column(nullable = false, length = 20)
private CourseStatus status;

private LocalDate launchDate;   // launch_date
```

Java	MySQL
String	varchar(255), or the length you set
LocalDate	date
enum, STRING	enum('ARCHIVED', 'DRAFT', ...)
enum, ORDINAL	a number

Never ORDINAL: reordering the enum silently changes the meaning of old rows.

Store enums by name and let dates be dates

Demo: An Entity



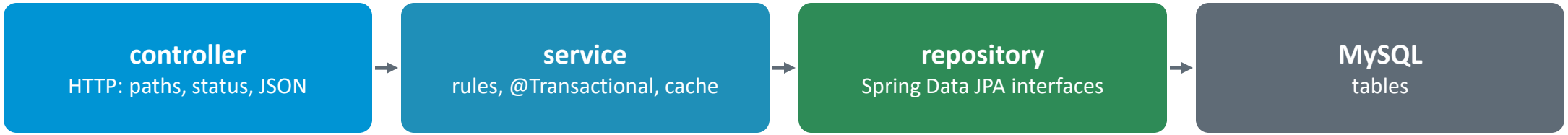
What we do

- **Map:** an entity with no detail.
- **Refine:** @Table, @Column, @Enumerated.
- **Look:** at the table in MySQL.

What it shows

- **DDL from a class:** no CREATE TABLE written.
- **Defaults:** 255 characters, enum as a number.
- **A better table:** names, NOT NULL, enum.

Layers of the API



Package	Holds
dto	CourseRequest, CourseResponse: records at the edge of the API
entity	Course, CourseStatus: the mapped classes, never returned to the client
exception	CourseNotFoundException and the ApiExceptionHandler that turns exceptions into responses

Rule: a controller never touches a repository or an entity.

Each layer has one job and calls only the layer below it

Repository Abstractions

Repository
marker interface

CrudRepository
save, find, delete, count

PagingAndSortingRepository
Pageable and Sort

JpaRepository
lists, flush

```
public interface CourseRepository extends JpaRepository<Course, Long> {
}
```

Two type arguments: the entity (Course) and the type of its id (Long).
No implementation: Spring Data builds it when the application starts.

An interface is the whole data layer

CRUD with JpaRepository

Operation	Endpoint	Repository	MySQL receives
Create	POST /api/courses	save(new Course(...))	INSERT INTO courses ... (201 + Location)
Read one	GET /api/courses/{id}	findById(id)	SELECT ... WHERE id = ?
Read all	GET /api/courses	findAll()	SELECT ... FROM courses
Update	PUT /api/courses/{id}	save(changedCourse)	SELECT, then UPDATE
Delete	DELETE /api/courses/{id}	deleteById(id)	DELETE ... WHERE id = ? (204)
Ask	(inside the service)	count(), existsById(id)	SELECT count(*) ...

Create, read, update and delete without SQL

Demo: CRUD Through the Layers

What we do

- **Create:** POST and the 201 Location.
- **Read:** GET by id and the list.
- **Change:** PUT and DELETE.

What it shows

- **Layers:** controller, service, repository.
- **DTOs:** the entity never leaves.
- **One save():** insert or update by id.

Validation of the Request

```
public record CourseRequest(
        @NotBlank @Size(max = 120) String title,
        @NotNull CourseStatus status,
        @NotNull LocalDate launchDate,
        @NotBlank @Size(max = 60) String category) { }

@PostMapping
public ResponseEntity<CourseResponse> create(@Valid @RequestBody
CourseRequest request)
```

Constraint	Rejects
@NotBlank	null, empty, spaces only
@NotNull	a missing value
@Size(max = n)	longer text

Dependency: spring-boot-starter-validation.
Without a handler: a bare 400 that does not say which field is wrong.

Reject bad input at the edge, before any service or SQL runs

Errors as Problem Details

```
@RestControllerAdvice
public class ApiExceptionHandler extends ResponseEntityExceptionHandler {

    @ExceptionHandler(CourseNotFoundException.class)
    public ProblemDetail notFound(CourseNotFoundException ex) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
                HttpStatus.NOT_FOUND, ex.getMessage());
        problem.setTitle("Course not found");
        return problem;
    }
}
```

Exception	Response
invalid body	400, errors per field
CourseNotFound	404
unknown sort field	400
PaymentFailed	402

Shape: status, title, detail, instance (+ errors) as application/problem+json.
Services throw, the handler decides: no status codes in the service.

One handler gives every error the same standard shape

Demo: Validation and Problem Details

What we do

- **Validate:** constraints and @Valid.
- **Report:** a problem detail with the field errors.
- **Replace:** the quick 404 by an exception.

What it shows

- **400 with details:** every wrong field at once.
- **404 everywhere:** GET, PUT and DELETE alike.
- **Not a 500:** an unknown sort field.

Derived Query Methods

```
List<Course> findByStatus(CourseStatus status);

List<Course> findByTitleContainingIgnoreCase(String text);

List<Course> findByLaunchDateBeforeAndStatusNot(
        LocalDate date, CourseStatus status);

long countByStatus(CourseStatus status);
boolean existsByTitle(String title);
```

Keyword	Meaning
Containing	LIKE %text%
IgnoreCase	case-insensitive
Before / After	date comparison
And / Or / Not	combine conditions

A typo fails early: findByStatuz stops the application at start-up.

The method name is the query

@Query: JPQL and Native SQL

```
@Query("select c from Course c " +
       "where c.category = :category " +
       "order by c.launchDate")
List<Course> jpqlByCategory(
        @Param("category") String category);
```

JPQL: entity and field names

```
@Query(value = "select * from courses " +
       "where category = :category " +
       "order by launch_date",
       nativeQuery = true)
List<Course> nativeByCategory(...);
```

Native: table and column names

Prefer: a derived method, then JPQL, then native SQL. **Parameters:** always bound, never concatenated.

When the method name gets too long or you need MySQL itself

Demo: Queries

What we do

- **Derive:** query methods as endpoints.
- **Mistake:** a typo in a method name.
- **Write:** JPQL and native SQL.

What it shows

- **No SQL typed:** the name is the query.
- **A start-up error:** not a runtime surprise.
- **The SQL:** with a bound ? parameter.

Pagination and Sorting

```
Page<Course> findByStatus(CourseStatus status, Pageable pageable);

// controller: Spring builds the Pageable from the URL
@GetMapping
public PageResponse<CourseResponse> list(Pageable pageable)

GET /api/courses?page=0&size=3&sort=launchDate
```

Statement	Why
SELECT ... LIMIT ?	the rows of the page
SELECT count(*) ...	the totals
ORDER BY	from the Sort

Page numbers start at 0. Sort in the database, not in Java.
Always sort a paged query: without an order the pages are not stable.

Never read a whole table to show ten rows

Demo: Paging

What we do

- **Page:** three courses at a time.
- **Totals:** elements and pages in the JSON.
- **Sort:** by two fields.

What it shows

- **Two statements:** rows and a count.
- **Stable pages:** a sort in every page request.
- **Database sorting:** ORDER BY in MySQL.

Transactions: All or Nothing

```
@Transactional
public void publish(Long id, boolean paymentFails) {
    Course course = courses.findById(id).orElseThrow();
    course.setStatus(PUBLISHED);
    courses.save(course);
    logs.save(new ActivityLog("Published course " + id));
    if (paymentFails) throw new PaymentFailedException(id);   // 402
}
```

Without	With @Transactional
each call commits	one commit at the end
a failure keeps half	a failure undoes all

Rolls back on: a RuntimeException (checked exceptions commit unless rollbackFor is set).
Where: on the service method that is one business operation.

One unit of work either happens completely or not at all

Demo: Transactions

What we do

- **Break:** a 402 in the middle.
- **Observe:** GET the status and the log.
- **Fix:** @Transactional.

What it shows

- **Half the work:** every call committed alone.
- **One unit:** all or nothing.
- **Rollback:** on a RuntimeException.

Caching a Read

Annotation	Effect
@EnableCaching	switches the caching annotations on (once, on a configuration class)
@Cacheable("courses")	the first call runs the method and stores the result; the next call with the same argument is answered from the cache
@CacheEvict(value = "courses", key = "#id")	removes the entry, so the next read goes to the database again

Good for: data that is read often and changes rarely.
Risk: a cache without eviction serves stale data; a simple in-memory cache is per application instance.

Answer repeated reads from memory

Demo: Caching

What we do

- **Cache:** GET by id with @Cacheable.
- **Count:** GET /api/stats after each read.
- **Evict:** after a rename.

What it shows

- **Zero statements:** the second read.
- **Fresh data:** an update evicts the entry.
- **Proof:** the statement count in /api/stats.

Key Takeaways

1. **ORM and connect**
   JPA, Hibernate and Spring Data JPA; three properties and a schema user.

2. **Entities**
   Annotations decide the table: names, NOT NULL, enums by name.

3. **Layers and CRUD**
   Controller, service, repository; DTOs at the edge; save, find, delete.

4. **Queries and paging**
   Derived names, JPQL, native SQL; pages and sorts in the database.

5. **Validation and errors**
   @Valid on the DTO; one handler returns problem details.

6. **Transactions and cache**
   @Transactional is all or nothing; @Cacheable saves reads, @CacheEvict keeps them fresh.



Happy Learning :)